

SIMULATING FALLING SNOW IN REAL-TIME ON THE GPU USING THE LATTICE BOLTZMANN METHOD

by

Marina Semenova

Submitted in partial fulfillment of the requirements
for the degree of Bachelor of Computer Science, Honours

at

Dalhousie University
Halifax, Nova Scotia
August 2026

© Copyright by Marina Semenova, 2026

*To my mother, who sponsored my adventures in higher education, and
my grandparents, whom I miss very much.*

Contents

Abstract	v
Acknowledgements	vi
Chapter 1 Introduction	1
Chapter 2 Background	3
2.1 Related Work	3
2.1.1 Wind Field Implementation	3
2.1.2 Hardware Acceleration	6
2.1.3 Snow Rendering	7
2.2 Snow Generation	8
2.2.1 Snow Generation Model	8
2.2.2 Modifications	10
Chapter 3 The Lattice Boltzmann Method	13
3.1 The D3Q19 Model	14
3.2 The Smagorinsky Turbulence Model	16
3.3 Boundary Conditions	18
3.4 Converting Between Physical Units and Lattice Units	18
3.5 Integration with Snow	19
3.6 GPU Acceleration	21
Chapter 4 Implementation	22
4.1 Model Inputs	22
4.1.1 Snow	22
4.1.2 Wind	23
4.1.2.1 Model Validation	24
4.2 Model Initialization	24
4.3 Pseudocode	24
4.4 Optimizations	26

4.4.1	Minimizing Memory Access	26
4.4.2	Increasing Data Coalescence	27
4.4.2.1	Structure of Arrays vs. Array of Structures	28
4.5	Rendering	30
Chapter 5	Experimentation	31
5.1	Method	31
5.2	Results	32
5.2.1	Varying Number of Particles	33
5.2.2	Varying Lattice Resolution	35
Chapter 6	Future Work	37
6.1	Towards Further Physical Accuracy	37
6.1.1	More Sophisticated Interactions	37
6.1.2	An Improved Wind Field	38
6.1.3	More Accurate Particle Updating	38
6.2	Towards Further High Fidelity Visualization	39
6.3	Optimization	39
6.4	Additional Experimentation	40
6.5	Testing Suite	40
Chapter 7	Conclusions	41
Bibliography		42

Abstract

Simulations of natural phenomena in the physical world, such as snowfall, are important in many fields, including computer graphics, gaming, and various engineering disciplines. This research seeks to implement physically accurate falling snow in real-time while maintaining high visual fidelity. This involves generating individual snow particles, accurately modelling wind via fluid dynamics, solving for particle motion, and rendering the simulation. To mimic the behaviour of snow in the physical world, the snow particles are generated as clumps of snowflakes arranged in concentric circles. The wind field is modelled using the Lattice Boltzmann Method, a method in computational fluid dynamics that models Boltzmann particle dynamics on a lattice, which avoids attempting to solve the Navier-Stokes equations directly. The computations of the physics, specifically the Lattice Boltzmann Method and the application of forces to snow particles, are accelerated through GPU computation using CUDA. Various optimizations are applied to improve coalesced memory access and, thus, running time. The three-dimensional simulation space is simultaneously rendered in real-time using OpenGL.

Acknowledgements

I would like to thank Dr. Alexander Brandt who helped make this project a reality and Dr. Stephen Brooks for agreeing to be the reader for my defense.

Chapter 1

Introduction

Simulations of natural phenomena in the physical world, such as snowfall, are important in many fields, including **computer graphics**, **game development**, and various engineering disciplines [1]. But, in these domains, snowfall which is both physically accurate and visually impressive is a rare sight [1]. Thus, this research sets out to create such a sight by combining the two in a truly **real-time** snowfall simulation using **OpenGL (Open Graphics Library)** — a flexible graphics library [2] — for its powerful rendering capabilities, and **CUDA (Compute Unified Device Architecture)** — a **parallel computing** platform for executing programs on a **Graphics Processing Unit (GPU)** [3] — to harness the massively parallel power of GPUs. This thesis delves deep into the topic of snowfall simulation and, after identifying an existing gap, seeks to fill it. It does this by applying a **Computational Fluid Dynamics (CFD)** method, the **Lattice Boltzmann Method (LBM)**, which is scarcely seen in snowfall simulation. Particularly, this method is well-known for readily admitting parallelism, and this research fully capitalizes on this fact. This, in combination with a **snow generation model** based on the physical properties of snow, produces a novel simulation and stimulates further exploration in this direction of inquiry. Particularly, areas that value the simulation’s real-time aspect, such as game development, **Augmented Reality (AR)**, and **Virtual Reality (VR)**, will benefit greatly from this research. It is worth noting that this research does not target a particular domain; its first and foremost goal is to combine physical accuracy with high-fidelity visualization, and performance is a secondary consideration. Measures will of course be taken to optimize the simulation, as this work does, but should improving the accuracy or the visualization result in poorer performance, this trade-off will be acceptable.

The thesis is organized as follows: Chapter 2 goes over the background for this work, Chapter 3 outlines the LBM, Chapter 4 discusses the implementation of the simulation, Chapter 5 describes the conducted experimentation, Chapter 6 prescribes future directions for this research, and, finally, Chapter 7 draws conclusions from this thesis.

Chapter 2

Background

This chapter evaluates work related to falling snow simulation and highlights the novelty of this research, as well as outlines the model used for snow generation and the manner in which it was modified in earlier work.

2.1 Related Work

Previous research in the area of snow simulation has mainly focused on snowfall accumulation, but the actual process of falling snow is far less studied [1]. Table 2.1 outlines existing snowfall implementations which are **physics-** and **particle-based**, and where the **wind field** is not constant. From this table, a research gap can be identified by establishing a unique combination of wind field implementation, hardware acceleration, and snow **rendering** not seen in any works listed. The following sections discuss each of these criteria and provide the rationalizations for the choices made to fill the research gap.

2.1.1 Wind Field Implementation

As can be seen, all but two models, Wang et al. [4] and Muraoka and Chiba [5], use **Navier-Stokes equations** to model the wind field, with most using simplified ones —also known as the **incompressible Euler equations** [6]. On the other hand, Wang et al. [4] uses the LBM, and Muraoka and Chiba [5] use a **vortex field**¹. All of these are **discretization methods** used in CFD.

This thesis uses the LBM to implement the wind field since it tends to be more physically accurate than using the Navier-Stokes equations [8, 9] and has been successfully used to model wind fields in similar domains [10, 11, 12, 13, 14].

¹A fluid field in which the flow revolves around an axis line [7].

	Wind Field Implementation	Hardware Acceleration	Snow Rendering
Herrera et al. [15]	Navier-Stokes equations [16, 17]	None	Particles
Eidissen [18]	Simplified Navier-Stokes equations [19, 20]	Parallelized on the GPU using CUDA by updating particles and storing the wind field as a three-dimensional texture on the GPU	Point sprites (billboards²)
Saltvik [19]	Simplified Navier-Stokes equations [21, 22, 6, 23, 24]	Parallelized the Successive Over-Relaxation (SOR) algorithm [25] used to solve the Poisson equation and rendered and ran wind computations in parallel [26] on a dual Central Processing Unit (CPU) system with and without Hyper-Threading³	Three rectangles joined at their centers to form axes Note: In a later publication [27], the Moeslund et al. [6] snow generation model is integrated
Wang et al. [4]	Lattice Boltzmann Method [10]	None	Billboards
Moeslund et al. [6]	Simplified Navier-Stokes equations [21, 22]	None	Triangles generated in layers of concentric spheres
Muraoka and Chiba [5]	Vortex fields [28, 29]	None	Multiple polygon shapes combined and used to generate a three-dimensional texture

Table 2.1: Previous work on physics- and particle-based snowfall where the wind field is not constant, sorted from most to least recent.

²A texture in a three-dimensional environment that always faces the camera. They are used as a rendering strategy to reduce resource use [30].

³Technology which allows a single **processing core⁴** to behave as if it were two cores [31].

⁴Essentially a processing unit [31].

The methods of implementing the physics of snow used in these works are not discussed as they are all relatively simple, but it is worth noting that there are prominent methods for modelling snow in other areas of snow simulation: the **Discrete Element Method (DEM)** (Figure 2.1) [32, 33, 34, 35], the **Material Point Method (MPM)** (Figure 2.2) [36, 37], **Fluid Implicit Particles (FLIP)** methods (Figure 2.3) [38], and **Smoothed-Particle Hydrodynamics (SPH)** (Figure 2.4) [39]. These are more sophisticated and powerful methods that are used when fine interactions need to be modelled and thus are deemed too computationally expensive for the general modelling of falling snow [1].

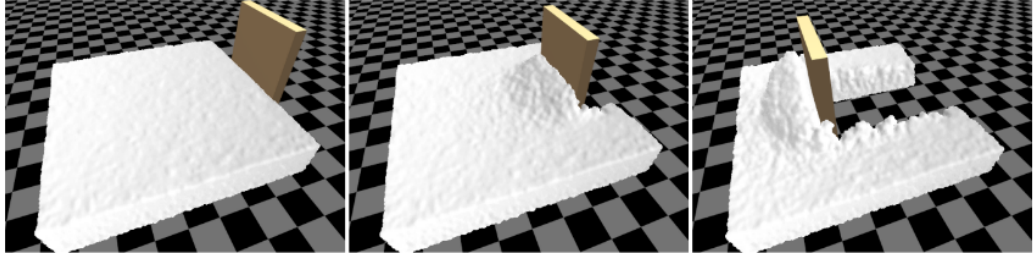


Figure 2.1: An example of using the Discrete Element Method (DEM) for snow simulation. Source: [34].

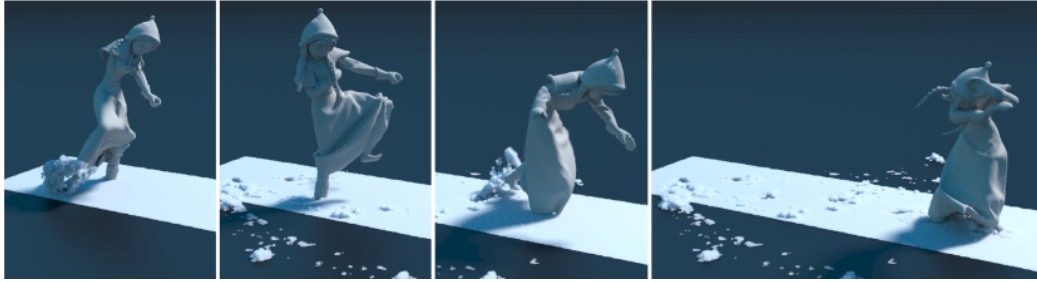


Figure 2.2: An example of using the Material Point Method (MPM) for snow simulation. Source: [37].

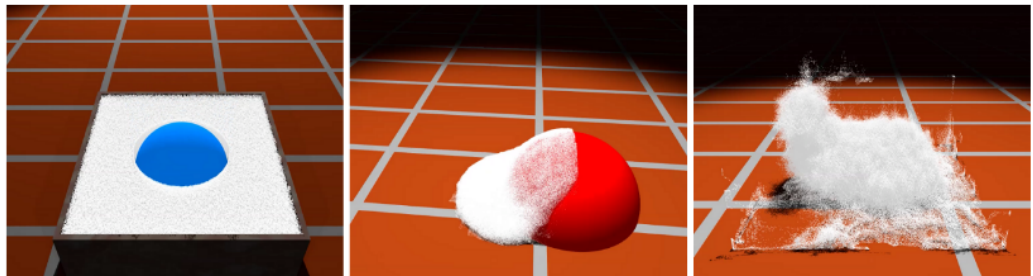


Figure 2.3: An example of using a Fluid Implicit Particles (FLIP) method for snow simulation. Source: [38].

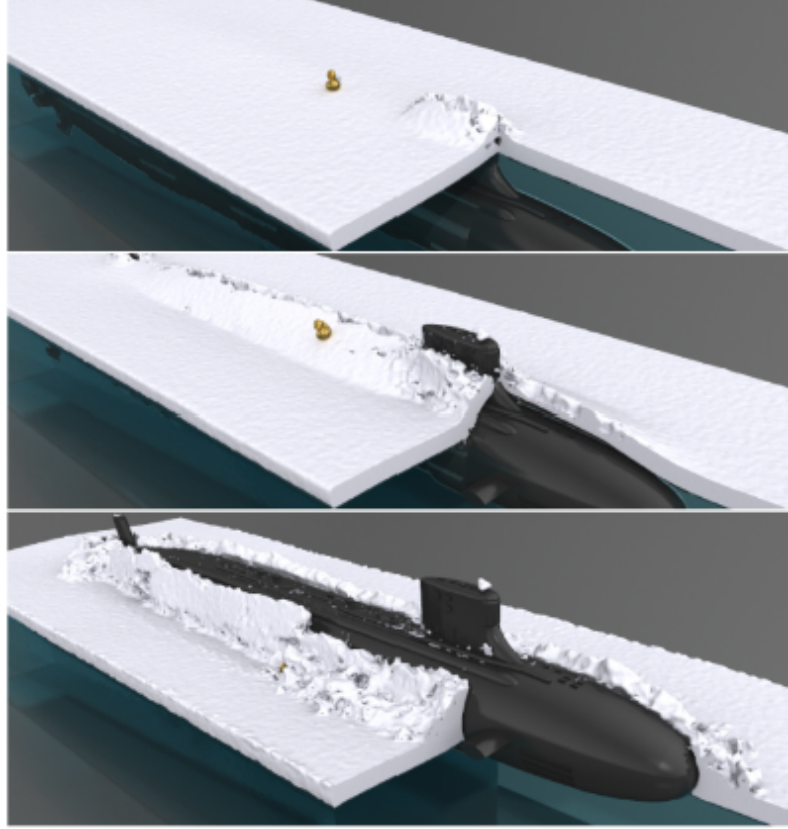


Figure 2.4: An example of using Smoothed-Particle Hydrodynamics (SPH) for snow simulation. Source: [39].

2.1.2 Hardware Acceleration

The two falling snow implementations which are accelerated — Eidissen [18] on the GPU and Saltvik [19] on a dual CPU system — both use Navier-Stokes equations. Consequently, the LBM was chosen for the wind field since the only implementation which used this method, Wang et al. [4], did not feature any hardware acceleration, and this choice would fill a gap in accelerated snowfall simulation. Furthermore, there is a large amount of existing work regarding its parallelization [40, 41], especially on the GPU [42, 43, 44, 45].

2.1.3 Snow Rendering

Another gap this work seeks to fill is to implement physically accurate snowfall while also seeking **high fidelity visualization**. Visual effectiveness tends to be ignored in physically accurate models [19, 4, 5], while high fidelity visualization of snow tends to disregard physical accuracy [46, 47]; for example, von Bernuth et al. [46] use a noise texture to add visual elements between particles to improve visual density, and Langer et al. [47] use random values to simulate wind field **turbulence**. While they produce visually impressive results, their simulation of snowfall has no basis in physics.

A snow generation technique that has been used in several snowfall models [27, 6] to attempt high fidelity visualization while maintaining physical accuracy is the model proposed by Moeslund et al. [6]. In this model, snow particles represent clumps of snow, which are created by generating triangles in layers of concentric spheres (Figure 2.5) [6]. The generation is also influenced by physical properties such as temperature, which is described in detail in Section 2.2 [6].



Figure 2.5: Moeslund et al.'s [6] model of snow generation. Source: [48].

Since Wang et al. [4] use very simple snow visualization — billboards — another gap which can be filled is to integrate a more sophisticated snow rendering technique, in this case that of Moeslund et al. [6], with the LBM. Moeslund et al. [6] was chosen due to its previous use in snowfall simulations [27, 6] and its existing implementation and evaluation in prior work related to this research.

There exist other snow generation techniques that also seek to combine physics and high fidelity visualization. Zou, Xie, and Zhao [49] modify Moeslund et al.’s [6] snow generation model by smoothing the triangles making up snow clumps using **Bézier curves**⁵. Muraoka and Chiba [5] imitate snow crystals by combining multiple polygon shapes and generating a three-dimensional texture from them. Kim and Lin [51] simulate the growth of ice crystals into physically accurate snowflakes, both on the CPU and on the GPU. Wong and Fu [52] model snow as a series of concentric, semitransparent, textured shells created by sampling randomly grown ice crystals.

Other examples of visually effective snow where physics are still largely considered can be found, such as the use of MPM by Disney in *Frozen* and other movies [36, 37]. These applications feature the more complex methods of modelling snow but they are not tailored to snowfall.

2.2 Snow Generation

The generation of snow in this work follows the method outlined by Moeslund et al. [6] with some modifications to improve the perceived realness of the snow.

2.2.1 Snow Generation Model

In the Moeslund et al. [6] model, triangles are generated in layers of **concentric spheres**, making each particle, or “snowflake”, a clump of snowflakes rather than a single crystal (Figure 2.5). The **diameter** of the clump of snow (D) is dictated by the temperature (Eq. 2.1), fluctuating randomly by $\pm 50\%$. The **density** ($\rho_{snowflake}$) is inversely proportional to the diameter (Eq. 2.2), and the proportionality constant $C_{humidity}$ is determined by the state of the snow, which is wet if the temperature is above -1°C or dry if it is below. It is not specified what state -1°C yields so wet snow is assumed. For wet snow $C_{humidity} = C_{wet} = 0.724 \frac{kg}{m^2}$ and for dry snow $C_{humidity} = C_{dry} = 0.170 \frac{kg}{m^2}$. It is noteworthy that the derived density is not used in the generation of snow clumps in the original method [6].

⁵A curve defined by control points [50].

$$D = \begin{cases} 0.015 |T|^{-0.35} & \text{if } T \leq -0.061 \\ 0.04 & \text{if } T > -0.061 \end{cases} \quad (2.1)$$

$$\rho_{snowflake} = \frac{C_{humidity}}{D} \quad (2.2)$$

Moeslund et al. [6] do not specify how many layers are generated so the value of 6 is extrapolated visually. There are 40 triangles per layer if the snow is wet and 10 if it is dry. The triangles in each layer except the first are generated by randomly choosing a **vertex** in the previous layer and using its **spherical coordinates**⁶ and the triangle it is part to determine the vertices of the new triangle. This is done by setting the first vertex of the new triangle to have the same θ and ϕ as the randomly chosen vertex, and a ρ of a point that lies within the triangle the random vertex is part of, which is also less than the ρ of the random vertex. The second condition for ρ is poorly specified because it may be that the random vertex has the smallest ρ in the triangle, so it is ignored. The other two vertices of the new triangle have a θ_{new} and ϕ_{new} of the first vertex's $\theta \pm \epsilon$ and the first vertex's $\phi \pm \epsilon$, respectively, where ϵ is the allowed angular change, set to 80° by Moeslund et al. [6]. The ρ_{new} for each of these two vertices is determined by randomly choosing a ρ between the previous layer and the next layer. It is not specified how the first layer is generated, so it is assumed that the first vertex is generated randomly with a ρ between 0 and the next layer, and the two other vertices are generated the same as in the other layers, but the θ_{new} and ϕ_{new} are of the first vertex.

⁶Coordinates with respect to the concentric spheres specified by θ , ϕ , ρ .

2.2.2 Modifications

An experiment was devised where the following modifications to the model were proposed:

- (a) Adding variance to the allowed angular change (ϵ) by having separate values for θ and ϕ , as well as by considering the state of the snow;
- (b) Setting a range of allowed change for the opacity of triangles;
- (c) Implementing a function for the number of layers dependent on density.

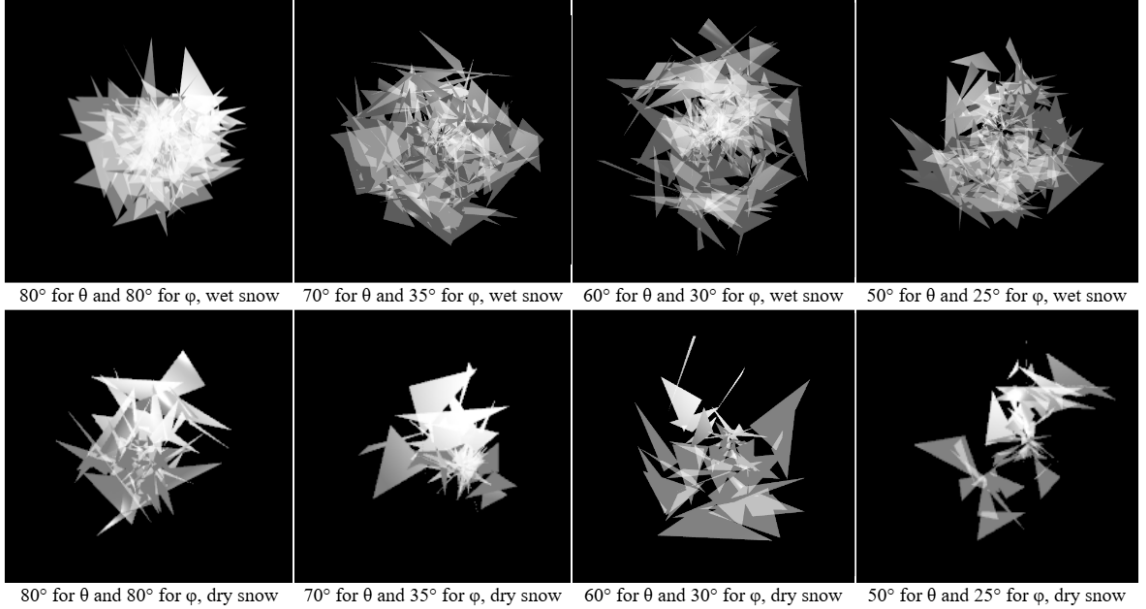


Figure 2.6: Comparisons for the allowed angular change in θ and ϕ .

For (a), reducing the allowed angular change and setting different yet proportional allowances for θ and ϕ yielded an improvement (Figure 2.6). For wet snow, 60° of allowance for θ and 30° of allowance for ϕ was best, and for dry snow, 70° for θ and 35° for ϕ was best. These values generated triangles which had a smaller difference between their width and height, resulting in a more realistic shape for a snowflake than those in Moeslund et al.'s [6] model and which were neither too crowded nor too spread apart.

For (b), adding a degree of randomness to the opacity of triangles improved the appearance of the snow, with a variance of $\pm 25\%$ for both wet and dry snow looking best (Figure 2.7). A higher variance yielded too much contrast to be realistic.

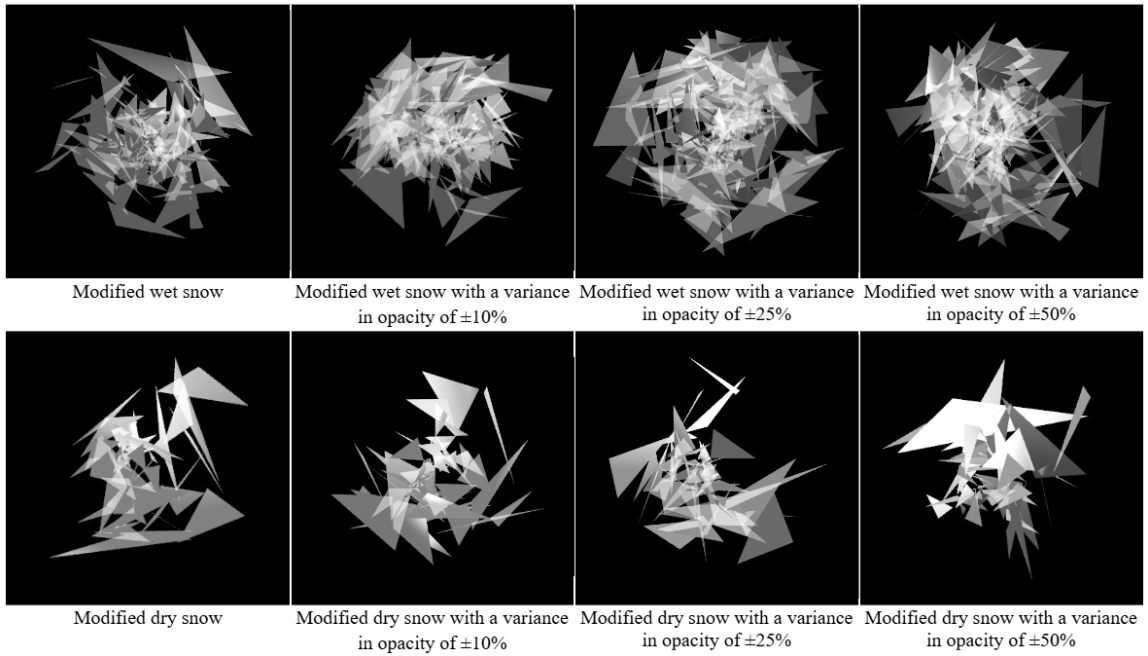


Figure 2.7: Comparisons for the variance in opacity.

For (c), 5 layers for wet snow and 6 layers for dry snow was best (Figure 2.8). A difference of layers between the two states of snow resulted in an improvement but any less or any more of each caused either crowding or sparseness.

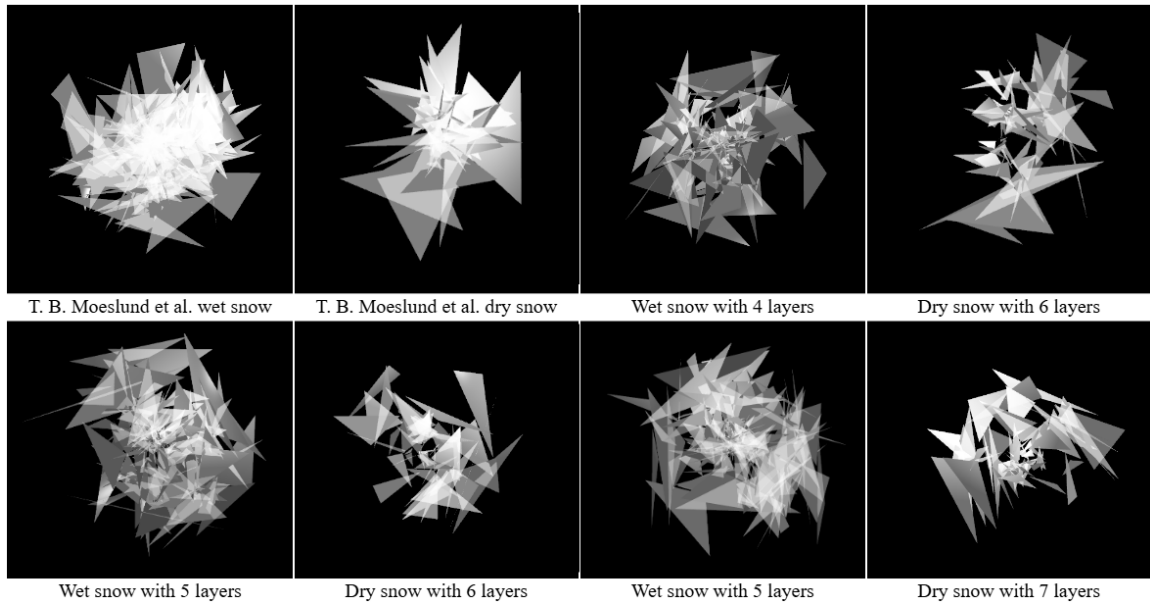


Figure 2.8: Comparisons for the number of layers for wet snow and the number of layers for dry snow.

Some limitations of this research include the difficult nature of evaluating results, as well as the great variation between generated particles.

Chapter 3

The Lattice Boltzmann Method

The LBM is a method for CFD that models **Boltzmann particle dynamics** on a **lattice**⁷ [54]. It originates from the **lattice gas automata method**⁸ and is essentially a discretization of the **Boltzmann equation** (Eq. 3.1), which describes the laws of the **distribution** of **particle clusters** rather than the movement of individual particles [42, 56, 4].

$$\frac{\partial f}{\partial t} + \vec{v} \cdot \nabla f = Q \quad (3.1)$$

Specifically, it describes how the average number of flow particles with a given velocity changes between neighbouring lattice points [10]. Thus, each lattice point is associated with **velocity distributions** that describe the **probability** of the presence of a flow particle with a given velocity [10]. In the LBM, the set of these velocities is defined by the number, orientation, and length of **lattice links**, and particles **stream** along these links from one point to another **synchronously** in **discrete time steps** [10]. In between time steps, the particles collide, and these collisions are typically modelled by the **Bhatnagar–Gross–Krook (BGK) model** [57], which represents them as a **statistical redistribution** of **momentum**, locally driving the system toward **equilibrium** [10, 56]. This follows from Boltzmann’s **H-Theorem**, which states that a system in **non-equilibrium** tends to its equilibrium state with maximal probability [8]. Furthermore, following **kinetic theory**, these collisions conserve **mass**, **momentum**, and **energy**.

⁷A regular arrangement of points in Euclidean — or physical — space [53].

⁸A type of discrete model of computation used to simulate fluid flows [55].

3.1 The D3Q19 Model

The LBM model can be described using the notation $\mathbf{D}d\mathbf{Q}q$, where d is the spatial dimension and q is the number of velocity distributions per lattice point. The model used in this work is D3Q19, which introduces more accuracy than Wang et al.'s [4] implementation of the D3Q15 model.

The **velocity vectors** in the D3Q19 model include the center with zero velocity in **sub-lattice 0**, the 6 **nearest-neighbour links** in sub-lattice 1, and the 12 **second-nearest links** in sub-lattice 2 (Figure 3.1). Each of these so-called **microscopic velocity vectors** are denoted by $\vec{e}_{qi}$ (Eq. 3.2). The “unit” velocity vectors are scaled by c , the **lattice speed**, defined momentarily. Each vector corresponds to a **velocity distribution function**, denoted f_{qi} , where q is the sub-lattice and i is a particular velocity vector in the sub-lattice [10].

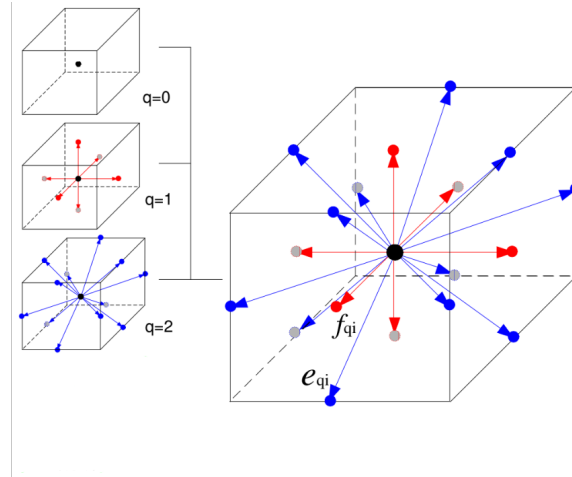


Figure 3.1: The D3Q19 Lattice Boltzmann Method lattice geometry for a lattice point, with the sub-lattices on the left and the combined D3Q19 lattice on the right. The velocity distribution for the associated velocity vector $\vec{e}_{qi}$ (Eq. 3.2) is denoted by f_{qi} . Source: [10].

$$\vec{e}_{qi} = c \cdot \begin{cases} (0, 0, 0) & \text{if } q = 0 \\ (\pm 1, 0, 0), (0, \pm 1, 0), (0, 0, \pm 1) & \text{if } q = 1 \\ (\pm 1, \pm 1, 0), (\pm 1, 0, \pm 1), (0, \pm 1, \pm 1) & \text{if } q = 2 \end{cases} \quad (3.2)$$

A fundamental property of the LBM is that the **macroscopic quantities** of a fluid can be recovered from the **microscopic velocity distributions** since the **evolution** of the distributions reflects the motion law of the fluid macroscopically [42, 4]. Two such quantities, density (ρ) and **velocity** ($\vec{u}$), at the lattice point $\vec{x}$ and time t are given by Eq. 3.3 and Eq. 3.4, respectively [10].

$$\rho(\vec{x}, t) = \sum_{qi} f_{qi}(\vec{x}, t) \quad (3.3)$$

$$\vec{u}(\vec{x}, t) = \frac{1}{\rho(\vec{x}, t)} \sum_{qi} f_{qi}(\vec{x}, t) \cdot \vec{e}_{qi} \quad (3.4)$$

The LBM can be broken down into two steps: streaming and collision [42]. In the streaming step, the distribution functions of each lattice point are streamed to their neighbouring nodes [42]. Next, in the collision step, the distribution functions are **relaxed** via the BGK **single-time-relaxation model** [57] towards a **local equilibrium**⁹ through collisions between the particles over a **characteristic time** τ — a constant representing the **relaxation time scale** — based on the updated macroscopic quantities at each lattice point [42, 10, 43]. This process is given by Eq. 3.5, where f_{qi} is the density of particles with velocity $\vec{e}_{qi}$ at the lattice point $\vec{x}$ and time t , f^{eq} is the **local equilibrium distribution function** given by Eq. 3.6, c is the lattice speed, w_{qi} is a weight coefficient, and $\Delta t = 1$ is the time increment in **lattice units** [42]. The lattice speed is given by $c = \frac{\Delta x}{\Delta t} = 1$, where $\Delta x = 1$ is the **lattice spacing**, or the lattice side length, in lattice units [42]. The **speed of sound** in lattice units (c_s) is $\frac{1}{\sqrt{3}}$; it is related to c by $c_s = \frac{c}{\sqrt{3}}$, thus constraining Δx and Δt [42]. In relation to the Boltzmann equation (Eq. 3.1), $\vec{v} = \vec{e}_{qi}$ and Q — the **collision integral** — is $\frac{\Omega}{\Delta t}$, where $\Omega = \frac{1}{\tau}(f^{eq} - f)$ is the BGK **collision operator** [56].

⁹The local balance of forces, in this case at a lattice point [58].

$$f_{qi}(\vec{x} + \vec{e}_{qi} \cdot \Delta t, t + \Delta t) = f_{qi}(\vec{x}, t) - \frac{1}{\tau} \left[f_{qi}(\vec{x}, t) - f_{qi}^{eq}(\vec{x}, t) \right] \quad (3.5)$$

$$f_{qi}^{eq}(\vec{x}, t) = \rho(\vec{x}, t) \cdot w_{qi} \cdot \left[1 + \frac{3}{c^2} (\vec{e}_{qi} \cdot \vec{u}(\vec{x}, t)) + \frac{9}{2c^4} (\vec{e}_{qi} \cdot \vec{u}(\vec{x}, t))^2 - \frac{3}{2c^4} \cdot \vec{u}(\vec{x}, t)^2 \right]$$

$$w_{qi} = \begin{cases} \frac{1}{3} & \text{if } q = 0 \\ \frac{1}{18} & \text{if } q = 1 \\ \frac{1}{36} & \text{if } q = 2 \end{cases} \quad (3.6)$$

3.2 The Smagorinsky Turbulence Model

The classic LBM is limited to modelling **steady flows** with low **Reynolds numbers**¹⁰ [42]. To simulate fluids with higher Reynolds numbers which are **dynamic** and **turbulent**, the LBM must be extended since Eq. 3.5 becomes **numerically unstable** as the **kinematic viscosity**¹¹, given by Eq. 3.7, tends to 0 [42]. This follows from the inability of the LBM to represent **flow dynamics** on scales smaller than the lattice spacing and the consequent inability to simulate **energy transfer** from larger scales to smaller ones that is needed to stabilize the model [61, 10]. Such an extension can be achieved by incorporating a **subgrid model**, such as the **Smagorinsky model** [62], to represent the physical effects that the **small-scale — or unresolved — motion** has on the **LBM-scale — or resolved — motion** [10].

$$\nu = \frac{2\tau - 1}{6} \quad (3.7)$$

$$\tau = 3\nu - \frac{1}{2}$$

¹⁰The ratio of inertial forces to viscous forces within a fluid. It is used to predict fluid flow patterns [59].

¹¹A measure of a fluid's internal resistance to flow under gravitational forces [60].

The Smagorinsky model [62] represents the influence of the unresolved motion by adding a **positive turbulent eddy viscosity** (ν_t) to the viscosity to get ν_{total} , abandoning the single-time-relaxation to allow the value of τ to vary over the lattice [10]. This viscosity is computed from the magnitude of the **local stress tensor**¹² $\bar{S}$ according to Eq. 3.8, where $0.01 \leq C \leq 0.05$ is the **Smagorinsky constant** — typically set to 0.04 [10] — and Δ is the scale at which the sub-grid model is used, which corresponds to the lattice spacing (Δx) [42].

$$\nu = C \cdot \Delta^2 \cdot |\bar{S}| \quad (3.8)$$

The magnitude of the local stress tensor is calculated using Eq. 3.9, where $\bar{\Pi}_{\alpha\beta}$ is the **local non-equilibrium stress tensor**, with α and β representing **Cartesian components** that run over the three spatial dimensions [42]. This calculation is a simplified version due to being paired with the LBM, notably avoiding **finite difference** computations [42].

$$|\bar{S}| = \frac{1}{C} \left[\sqrt{\nu^2 + 18C^2 \cdot \sqrt{\sum_{\alpha\beta} \bar{\Pi}_{\alpha\beta} \cdot \bar{\Pi}_{\alpha\beta}}} - \nu \right] \quad (3.9)$$

$$\bar{\Pi}_{\alpha\beta} = \sum_{qi} \vec{e}_{qi_\alpha} \cdot \vec{e}_{qi_\beta} \cdot (f_{qi} - f_{qi}^{eq})$$

The local variation of τ (τ_S) is then computed for a given lattice point by Eq. 3.10 [10].

$$\tau_S = 3\nu_{total} + \frac{1}{2} = 3(\nu + \nu_t) + \frac{1}{2} = 3(\nu + C \cdot \Delta^2 \cdot |\bar{S}|) + \frac{1}{2} \quad (3.10)$$

A subgrid model has not been integrated into this research at this point but is a high-priority next step which would enable the precise modelling of physics.

¹²Represents the internal distribution of forces at a point within a continuous medium [63].

3.3 Boundary Conditions

Boundary conditions in the LBM define the unknown distribution functions at the “boundary”, in this case the distributions of cells beyond the defined volume [42]. There are many ways to implement boundary conditions; this work implements **bounce-back boundary conditions** in combination with an **inlet**¹³ plane to stimulate the wind field.

For a bounce-back boundary, if $\vec{x} + \vec{e}_{qi}$ from Eq. 3.5 goes beyond the lattice volume, $f_{qi}(\vec{x}, t)$ is set to $f_{qi'}(\vec{x}, t)$, where i' is an i such that $\vec{e}_{qi'} = -\vec{e}_{qi}$, effectively streaming particles back in the direction they came from [65]. This simulates a solid boundary with a **no-slip condition**¹⁴ [65].

To create an inlet plane, all lattice points which lie on the plane are set to a constant velocity [67].

3.4 Converting Between Physical Units and Lattice Units

The LBM is defined in terms of lattice units, so **physical units** need to be converted to lattice units before they can be used in the LBM [68]. Such a conversion follows from **dynamic similarity**, where, by defining **characteristic scales** for units involved in a conversion — in this case physical and lattice time and space — rescaling them to be **dimensionless** is possible; if dimensionless quantities, such as the Reynolds number (**Re**) and the **Mach number**¹⁵ (**M**) are equal in both unit systems, the ratio of their characteristic scales can be used to convert between them [68].

In the LBM, the **lattice spatial characteristic scale**, Δx , which is the length of a cell, and the **lattice characteristic temporal scale**, Δt , are often set to 1 for simplicity [68]. In physical space, the spatial characteristic scale, $\Delta x'$, can be similarly defined by the cell length, which is calculated from the lattice side length in the simulation and the **lattice resolution**, or the lattice side length [68]. The physical time characteristic scale, $\Delta t'$, is derived from the speed of sound, c_s' , and $\Delta x'$ [68].

¹³A boundary where a fluid enters a system [64].

¹⁴At a boundary, a fluid’s velocity is equal to the velocity of the boundary [66].

¹⁵A dimensionless quantity representing the ratio of flow velocity past a boundary to the local speed of sound [69].

Then, conversion from physical spatial units to lattice units can be achieved through dividing by $\frac{\Delta x'}{\Delta x}$, which reduces to $\Delta x'$ [68]. Similarly, conversion from physical time units to lattice units can be achieved through dividing by $\frac{\Delta t'}{\Delta t} = \Delta t'$ [68]. Converting from lattice units to physical units involves dividing by the reciprocals of the above [68].

For spatial units, when implementing the LBM the lattice points are often made equal to indices of a 3-dimensional array, and therefore physical values must be translated before conversion such that the physical volume matches the lattice space [42].

3.5 Integration with Snow

Snowflakes are incorporated into the LBM as particles with a single position in three-dimensional space to represent the entire snowflake. Following a simplified model, they are influenced by three forces: **gravity**, wind, and interactions between particles, including **cohesion**, **fragmentation**, and **compaction** [4, 34]. In this context, gravity is the downward trajectory in the absence of wind, which also accounts for forces other than gravity but which have significantly less effect than the wind and therefore allow simplifying assumptions to be made. These forces include **drag**¹⁶, **buoyancy**¹⁷, and **lift**¹⁸ [6]. Modelling the interactions between snow particles is out of scope but has been identified as a fundamental direction for future research.

Assuming the effects of **inertia**¹⁹ are neglected, a snow particle $\vec{x} = (x_x, x_y, x_z)$ — in lattice units after conversion from physical units — will be subjected over Δt to the velocity of the wind at its corresponding lattice point ($\vec{u}(\vec{x}, t)$) and gravity in lattice units ($-\vec{u}_g$) as described by Eq. 3.11 [4].

$$\vec{x}_{new} = (\vec{x}_x + \vec{u}(\vec{x}, t)_x \cdot \Delta t, \vec{x}_y + [\vec{u}(\vec{x}, t)_y - \vec{u}_{g_y}] \cdot \Delta t, \vec{x}_z + \vec{u}(\vec{x}, t)_z \cdot \Delta t) \quad (3.11)$$

¹⁶A force acting opposite to the direction of movement of an object in a fluid [70].

¹⁷The force exerted by a fluid opposing the weight of an object in it [71].

¹⁸Lift is the component of the force exerted on an object by a fluid flowing around it that is perpendicular to the flow direction [72].

¹⁹The natural tendency for objects in motion to stay in motion and objects at rest to stay at rest in the absence of external forces [73].

If the position of the snow particle does not coincide with any point on the lattice, the wind velocity at that point can be determined by **linear interpolation** [4]. By looking at a snow particle's 8 nearest lattice points, whose locations are denoted $\vec{x}_i = (x_{i_x}, x_{i_y}, x_{i_z})$ for $0 \leq i < 8$ (Figure 3.2), the velocity of the wind field at $\vec{x}$ can be approximated by Eq. 3.12 [4].

As this is a relatively simple way to update particle positions, future work should certainly look to replace it with a more complex method, such as a **second-order Runge-Kutta integration** [6].

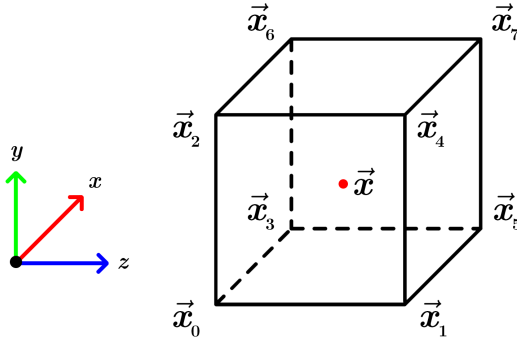


Figure 3.2: The assignment of the 8 lattice points nearest to a snow particle's location ($\vec{x}$) [4].

$$\begin{aligned}
 \vec{u}(\vec{x}, t) = & (1 - \delta x) \cdot (1 - \delta y) \cdot (1 - \delta z) \cdot \vec{u}(\vec{x}_0, t) \\
 & + (1 - \delta x) \cdot (1 - \delta y) \cdot \delta z \cdot \vec{u}(\vec{x}_1, t) \\
 & + (1 - \delta x) \cdot \delta y \cdot (1 - \delta z) \cdot \vec{u}(\vec{x}_2, t) \\
 & + \delta x \cdot (1 - \delta y) \cdot (1 - \delta z) \cdot \vec{u}(\vec{x}_3, t) \\
 & + (1 - \delta x) \cdot \delta y \cdot \delta z \cdot \vec{u}(\vec{x}_4, t) \\
 & + \delta x \cdot (1 - \delta y) \cdot \delta z \cdot \vec{u}(\vec{x}_5, t) \\
 & + \delta x \cdot \delta y \cdot (1 - \delta z) \cdot \vec{u}(\vec{x}_6, t) \\
 & + \delta x \cdot \delta y \cdot \delta z \cdot \vec{u}(\vec{x}_7, t) \\
 \delta x = & \frac{\vec{x}_x - \vec{x}_{0_x}}{\Delta x} \\
 \delta y = & \frac{\vec{x}_y - \vec{x}_{0_y}}{\Delta y} \\
 \delta z = & \frac{\vec{x}_z - \vec{x}_{0_z}}{\Delta z}
 \end{aligned} \tag{3.12}$$

3.6 GPU Acceleration

Another significant aspect of the LBM is that it readily admits parallelization, which enables incredible speedup by leveraging GPUs [42, 43, 44, 45]. This is a result of being able to update each lattice point independently and thus being able to do so at the same time. The methodology followed in this work for implementing the LBM on the GPU, specifically using CUDA, is based on that of Delbosc et al. [42]. Strategies used include minimizing memory accesses and increasing **data coalescence**, further discussed in Chapter 4.

Chapter 4

Implementation

This simulation was implemented in `C++`, using OpenGL to visualize the simulation, and using CUDA to parallelize computations on the GPU for massive speedup. The source code is publicly available at <https://github.com/marssemenova/SnowVisualizer>. The remainder of the chapter elaborates upon the logistics of the implementation, particularly the model parameters and initialization, the methodology, and the high-level **pseudocode**.

4.1 Model Inputs

The following sections detail the parameters provided which configure the simulation.

4.1.1 Snow

The snow renderer module takes the number of particles, the temperature of the environment in $^{\circ}\text{C}$, and the extents of the simulation space in centimetres — to match the existing snow generation model — in the x, y, and z dimensions of world space as inputs. Due to the nature of the LBM and for simplicity, the extents are adjusted such that the range in each dimension is equal. The range is set to the minimal range in the extents, and the higher value in a dimension’s extent will be trimmed to match this range as needed. These parameters are then used to generate an initial scene of snowflakes.

4.1.2 Wind

The wind field is configured by the input of the **macroscopic flow speed** of the wind (u') per $\Delta t'$ in kilometres per hour and the lattice resolution (N), along with the temperature of the environment and the extents used in the snow generation. The extents and the lattice resolution are used to calculate $\Delta x'$, and the effect of temperature on the properties of air is used to derive c_s' [74]. Due to the data available for the speed of sound at certain temperatures, the simulation can only model temperatures between -25°C and 35°C . The physical time unit ($\Delta t'$) is determined by Eq. 4.1 from $\Delta x'$, c_s' , and c_s [68].

$$\Delta t' = \frac{c_s}{c_s'} \cdot \Delta x' \quad (4.1)$$

Lastly, while τ should be calculated according to Eq. 4.2 from $\Delta t'$, $\Delta x'$, and ν' — which is derived from the effect of temperature on the properties of air [75] — it is set to an arbitrary value — 0.55 — which yields stability [68]. This is a limitation of the model resulting from a lack of subgrid model, such as the Smagorinsky turbulence model described in Section 3.2; the low viscosity of air would cause τ to be too close to 0.5 to maintain stability [75, 42]. Another direct consequence of this is that, since the model would not be able to simulate such fast flow, u' and $-\vec{\mathbf{u}}_g$ are set arbitrarily so that the visualization imitates wind, and c_s' is scaled as well. Thus, these values do not produce the conditions they would in the physical world.

$$\tau = 3\nu + \frac{1}{2} = 3 \left[\nu' \cdot \frac{\Delta t'}{(\Delta x')^2} \right] + \frac{1}{2} \quad (4.2)$$

4.1.2.1 Model Validation

Once the simulation has been expanded to enable air to be accurately represented by the parameters by implementing a subgrid model, the inputs provided would be validated by checking if the conditions they describe result in a stable system. The Reynolds number ($\mathbf{Re}$) and the Mach number ($\mathbf{M}$) are calculated by Eq. 4.3 and Eq. 4.4, respectively, and the two conditions to satisfy in order to maintain stability are given in Eq. 4.5, where $\mathbf{M}_{max}$ is some predetermined threshold much less than 1 so as to maintain the incompressibility assumption for the fluid [68].

$$\mathbf{Re}' = \mathbf{Re} = \frac{u'N'}{v'} \quad (4.3)$$

$$\mathbf{M}' = \mathbf{M} = \frac{u'}{c_s'} \quad (4.4)$$

$$\begin{aligned} \frac{\mathbf{Re}}{N} &\leq 24 \\ \mathbf{M} &\leq \mathbf{M}_{max} \ll 1 \end{aligned} \quad (4.5)$$

4.2 Model Initialization

Before the simulation can run, the wind field has to be configured to some **initial state** from which it will evolve. This is done by setting all the f_{qi} of each lattice point to f_{qi}^{eq} where $\rho(\vec{x}, t) = 1$ and $\vec{\mathbf{u}}(\vec{x}, t) = (u, 0, u)$ [67]. It is worth mentioning that these parameters are adjustable.

4.3 Pseudocode

Algorithm 1 outlines the high-level pseudocode of the simulation.

Algorithm 1: High-Level Pseudocode

- Input:** number of particles, extents (cm), temperature ($^{\circ}\text{C}$), macroscopic flow speed of the wind (u') (km/h), lattice resolution (N)
- 1 Generate the initial snow scene using the snow generation algorithm with the number of particles, extents, and temperature as the inputs.
 - 2 Initialize data on the GPU (extents, snowflake data²⁰, source lattice, destination lattice, N , $\Delta x'$, $\Delta t'$, u , f_{qi}^{eq} *init*).
 - 3 Set the initial conditions for the source lattice and save the local equilibrium distribution function (f_{qi}^{eq}) values as f_{qi}^{eq} *init*.
 - 4 **While running, for each frame:**
 - 5 **Run the LBM kernel:**
 - 6 **If a lattice point is on the inlet plane (minimal yz-plane):**
 - 7 Calculate the updated distribution functions (f_{qi}) at the new time step where $f_{qi}^{eq} = f_{qi}^{eq}$ *init* and save them to the current lattice point.
 - 8 **Otherwise:**
 - 9 **Perform the streaming step:** stream 19 f_{qi} to the current lattice point from adjacent ones.
 - 10 Apply bounce-back boundary conditions.
 - 11 Calculate the macroscopic density (ρ) and velocity ($\vec{u}$).
 - 12 Calculate f_{qi}^{eq} for each qi .
 - 13 **Perform the collision step:** calculate the updated f_{qi} at the new time step and save them to the current lattice point.
 - 14 **Run the snow particle update kernel:** Calculate the new position of each snow particle according to applied forces and save the offset.
 - 15 **If a particle is out of bounds in the y direction:**
 - 16 Respawn the particle in a random position at the top of the volume.
 - 17 **If a particle is out of bounds in the x or z direction:**
 - 18 Negate the calculated offset in the direction headed out of bounds.
 - 19 **Run the vertices update kernel:** Update the vertices of the snow using the offsets from the snow particle update kernel.
 - 20 Copy the updated vertices from the GPU.
 - 21 Swap the source lattice and the destination lattice.
 - 22 Update the **Vertex Buffer Objects (VBOs)**²¹ used in modern OpenGL rendering.
 - 23 Render new frame.

²⁰Includes the vertices, the number of particles, the array used to pass offsets between kernels, and for each snow particle its position, the number of polygons in the snow clump, and the index of the particle's first vertex in the vertices array.

²¹An object which stores vertex data, such as positions, normals, and colours, on the GPU.

4.4 Optimizations

The primary speedup this research achieves is done by leveraging the GPU to parallelize computations. For the computation of the LBM, each **thread of execution** — which runs in parallel with other threads — updates a single lattice point. Calculating the change in position according to the application of forces to snow particles involves each thread handling a **batch** of particles to optimize thread efficiency. When updating the actual vertices that make up the snow clumps represented by single-point particles, each thread applies the stored offset to each vertex in a snow particle’s **geometry**.

This section details various optimizations implemented throughout the simulation, many of which are prescribed by Delbosc et al. [42].

4.4.1 Minimizing Memory Access

The main drawback of the LBM on the GPU is **memory bandwidth**, or the rate at which data can be read from or written to memory [42, 76]. Delbosc et al. [42] increase it by reducing the number of accesses to global memory by a factor of 2, achieved through merging the streaming and collision steps into one GPU **kernel** — simply a function executed on the GPU [3]; instead of two writes and two reads for each f_{qi} , the merged kernel only performs one read and one write (Figure 4.1) [42].

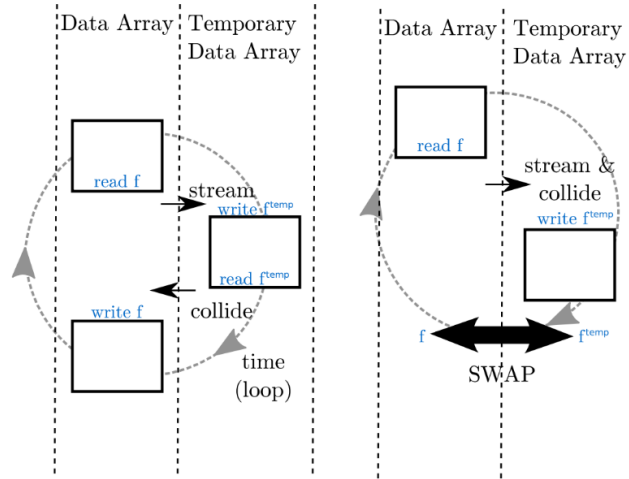


Figure 4.1: Comparison between the unmerged (left) and merged (right) kernels. Source: [42].

Moreover, special attention was paid to what data structures each kernel reads and writes from, and if possible, kernels were split up so as to maximize the efficiency of memory accesses. A result of this was the separation of the offset calculation, now done in the snow particle update kernel, and the offset application, now done in the vertices update kernel. Conversely, **thread coarsening** involves each thread processing more than one computation that would otherwise be handled by a single thread [77]. This is employed in the snow particle update kernel — a **memory-bound kernel** where work per thread is low — to optimize memory access.

Another strategy is to keep all data on or easily accessible by the GPU without needing to copy between the GPU and the CPU. At this point, this is only followed by using CUDA’s `cudaHostAlloc()` function to “pin” CPU memory, allowing the GPU to access it more efficiently, and maintaining copies of often-accessed data, such as snowflake data, on the GPU instead of copying it back and forth, but additional improvements are discussed in the future work section [78].

4.4.2 Increasing Data Coalescence

Memory bandwidth can be further increased by ensuring memory accesses are **coalesced**, meaning threads in a **block** — a grouping of threads that are executed simultaneously on a **Streaming Multiprocessor (SM)**²² — access consecutive memory locations [42, 80]. This allows the memory to be accessed in a single request, minimizing resource use [42].

Delbosc et al. [42] identify the streaming step as a part of the LBM algorithm which prevents it from being fully coalesced, thus creating a **bottleneck**, and they suggest a means of mitigating this [42]. Since uncoalesced reads are faster than uncoalesced writes, the streaming step can be accelerated by using the **pull-in method**, where the post-collision distribution functions are pulled from the neighbouring nodes to the center node, instead of the **push-out method**, where the post-collision distribution functions are pushed from the center node to the adjacent nodes, to stream the distribution functions [42].

²²Roughly analogous to the processing cores of CPUs [79].

In the snow particle update kernel, data coalescence is also considered when determining which particles are processed in a batch, ensuring that the data accessed by threads running in parallel is stored sequentially in memory.

4.4.2.1 Structure of Arrays vs. Array of Structures

A modification that results in data coalescence is using the **Structure of Arrays (SoA) scheme** (Listing 4.2), which is better for memory access than the contending **Array of Structures (AoS) scheme** (Listing 4.1) [42].

```

1 // group 19 float velocity distributions together
2 struct LatticePoint { float f0, f1, ..., f18; }
3 // create an array of N*N*N lattice points
4 LatticePoint Lattice[N*N*N];
5 // access f0 at the lattice point (x, y, z)
6 float f0 = Lattice[x+y*N+z*N*N].f0;

```

Listing 4.1: Array of Structures scheme [42].

```

1 // group 19 velocity distribution float arrays of size N*N*N
  together
2 struct Lattice {
3     float f0[N*N*N];
4     float f1[N*N*N];
5     ...
6     float f18[N*N*N];
7 }
8 // access f0 at the lattice point (x, y, z)
9 float f0 = Lattice.f0[x+y*N+z*N*N];

```

Listing 4.2: Structure of Arrays scheme [42].

With the AoS memory access pattern (Figure 4.2), threads do not access consecutive memory locations and thus cannot be grouped into a single memory access to optimize the process [42]. On the other hand, the SoA memory access pattern (Figure 4.3) remedies this and is therefore the favourable option [42].

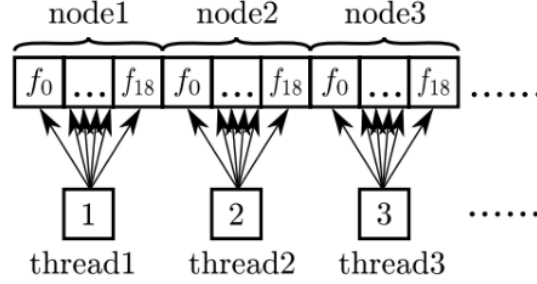


Figure 4.2: Memory access pattern for an Array of Structures. Source: [42].

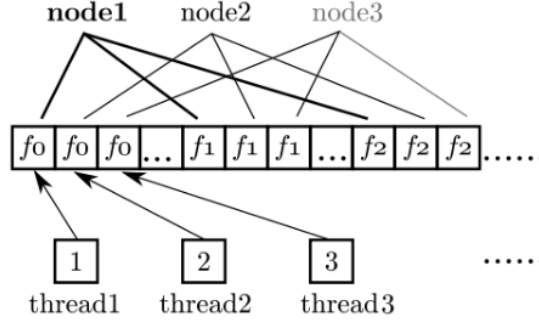


Figure 4.3: Memory access pattern for a Structure of Arrays. Source: [42].

This scheme is made even more effective by matching the layout of the memory with the layout of the threads [42]. This is done by storing each of the 19 velocity distribution arrays as one-dimensional arrays which are first aligned in the x-direction, then in the y-direction, and finally in the z-direction [42]. To compliment this, the threads are structured such that they match the volume of the lattice; they are aligned along a one-dimensional block of size N and the blocks of threads are aligned on a two-dimensional **grid**²³ of size (N, N) [42]. Listing 4.3 shows how each f_{qi} is accessed by threads in the LBM kernel.

²³Of thread blocks [79].

```

1 // access the thread indices within the thread layout
2 int x = threadIdx.x;
3 int y = blockIdx.x;
4 int z = blockIdx.y;
5 // compute the 1D index corresponding to (x, y, z)
6 int index = x + y * N + z * N*N;
7 // access f0 at the lattice point (x, y, z)
8 float f0 = Lattice.f0[index];

```

Listing 4.3: Indexing into the one-dimensional velocity distribution arrays [42].

4.5 Rendering

The rendering of the falling snow simulation is done using modern OpenGL. Modern OpenGL — version 3.0 or later — introduces the use of programmable **shaders**, as well as VBOs and **Vertex Array Objects (VAOs)**²⁴, improving GPU efficiency. Due to the emphasis on the real-time performance of the simulation, high-fidelity visualization was largely not considered in this work despite being a primary goal of this research. An instance where it is considered is the snow generation previously discussed in Section 2.2, where the snow particles are generated as clumps of snow according to a modified version of Moeslund et al.’s [6] algorithm. The shader used to render the simulation follows the standard **Phong model**²⁵.

²⁴Store the format of vertex data as well as buffer objects providing the vertex data [81].

²⁵A common model for shading which incorporates ambient lighting, direct lighting, and shininess [82]. Although fast and satisfactory for many cases, it is relatively simple [82].

Chapter 5

Experimentation

The implementation described in the previous chapter was evaluated by gauging the additional computational power that acceleration on the GPU provided. This was done by **profiling** the simulation’s performance when the number of particles increased, as well as when the lattice resolution increased. Below, the methodology is outlined and the results are discussed.

5.1 Method

The primary goal of this experimentation was to evaluate how far this simulation can be pushed before it is no longer real-time. Three target thresholds were established: **30 FPS (Frames Per Second)**, or 33.33 milliseconds per frame, 60 FPS, or 16.67 milliseconds per frame, and 90 FPS, or 11.11 milliseconds per frame. These were all selected as they are the most standard frame rate values used in practice [83, 84]. Ideally, the simulation should be able to maintain 60 FPS despite heavy computational costs.

For each frame, the profiling was split between time spent in CUDA and time spent rendering. Time spent in CUDA was further split into time spent executing each kernel and time spent transferring the updated vertices from the GPU to the CPU. The reported times were an average over 100 frames. The percentage of time taken up by CUDA in comparison to that taken up by rendering was also of interest.

Additionally, the percentage of GPU memory and the percentage of CPU memory used was recorded. The GPU memory percentage was measured using the `nvidia-smi pmon` command, and the CPU percentage was measured using the Windows Task Manager.

The first experiment involved varying the number of particles to determine at what number of particles each of the FPS thresholds can be expected. The values for the number of particles were 10,000, 20,000, 30,000, 40,000, and 50,000. The lattice resolution was fixed at 20.

Next, the lattice resolution was modified for the same reasons. Profiling was done for a lattice resolution of 20, 40, 80, 120, 160, 200, and 240, and the number of particles was fixed at 10,000.

The experiments were executed on an ASUS ROG Strix G15 Gaming Notebook housing an NVIDIA GeForce RTX 3060 Laptop GPU with 6 GB of GDDR6 VRAM, an AMD Ryzen 7 6800H CPU at 3.20 GHz, and 16 GB of DDR5 RAM at 4800 MT/s. The operating system used was Windows 11, version 25H2. The C++ standard was C++17, the GPU code was compiled with version 13.1.80 of the CUDA toolkit, and OpenGL 4.0 was used. Some additional system configuration was applied by Armoury Crate, where the operating mode was set to “performance” and the display settings were set to “racing”.

First, suitable initial conditions were set. The temperature was -5°C , the extents — in centimetres — were defined by $(-100, -100, -100)$ and $(100, 100, 100)$, u' was 0.05 kilometres per hour, $\vec{u}_g'$ was -9.81 centimeters per second squared, and c_s' was scaled by a factor of $\frac{1}{100}$.

5.2 Results

Below, the profiling results are showcased, analyzed, and discussed for both experiments. It is worth noting that these results are highly dependent on the hardware on which the experiments were executed.

5.2.1 Varying Number of Particles

Figure 5.1 reports the profiling for varying the number of particles. Overall, increasing the number of particles decreases FPS linearly. Additionally, the percentage of time spent in CUDA increases as the number of particles increases, implying that the computational cost of updating the vertices, particularly the cost of the vertices update kernel and copying the updated vertices from the GPU to the CPU, scales faster than render time. It can be interpolated that the simulation remains real-time for up to just under 50,000 particles, but this may not be the case if the lattice resolution is also increased.

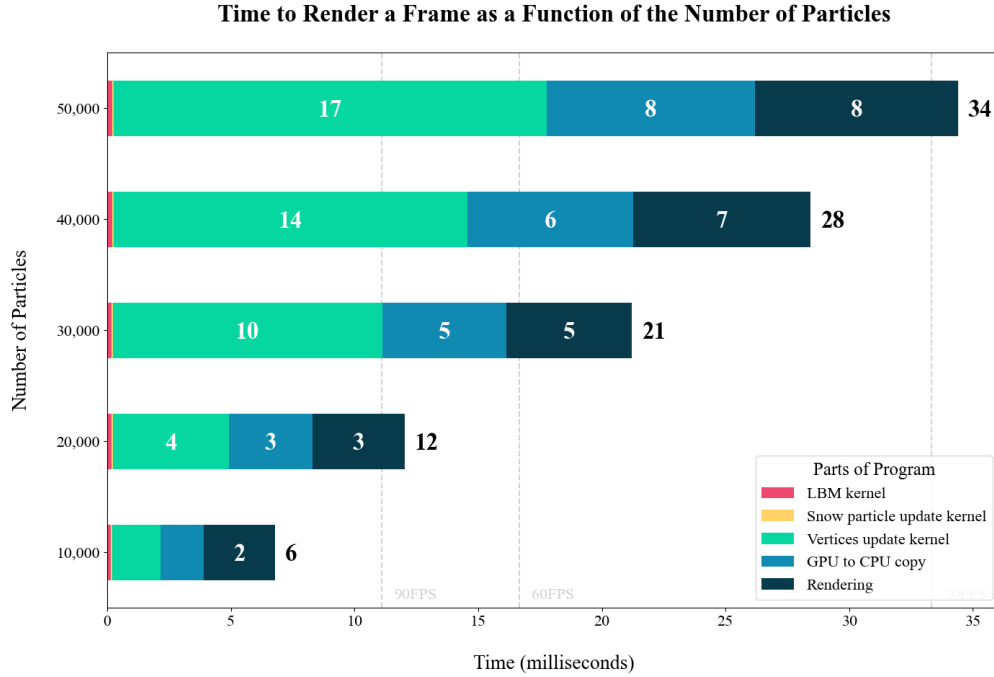


Figure 5.1: A stacked bar graph which presents the time it takes to render a frame, in milliseconds, for different amounts of particles. The total time is further split into separate time segments for various parts of the program.

Looking at memory use percentages, they remain fairly low across the board, leaving resources for other tasks. The CPU memory usage increases linearly; on the other hand, GPU memory usage increases linearly then stabilizes around 16, which is worth exploring.

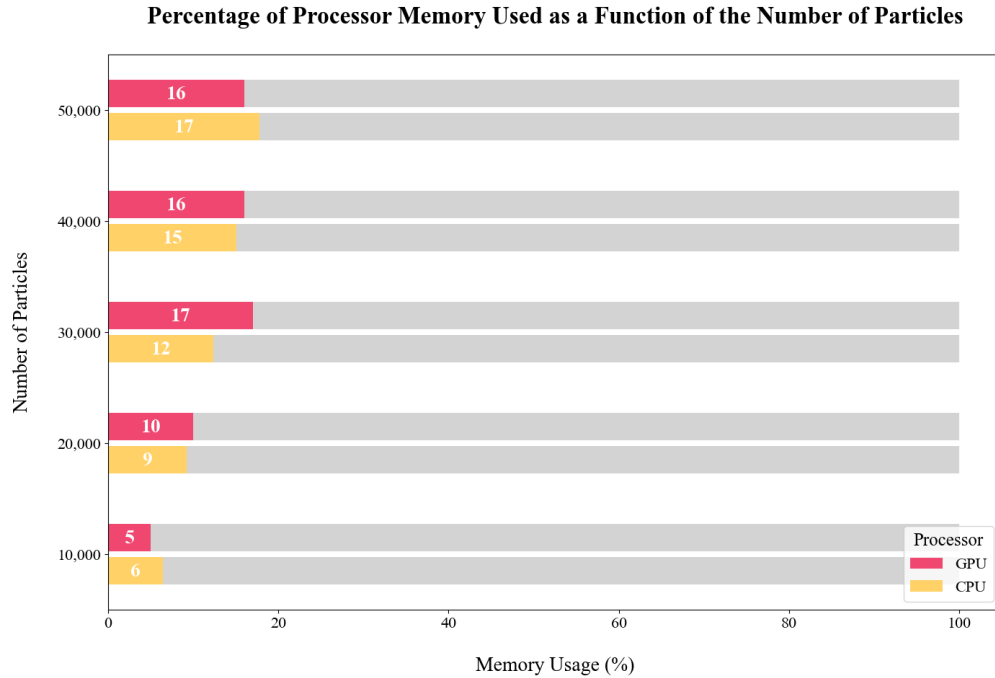


Figure 5.2: A stacked double bar graph which presents the percentage of GPU and CPU memory used for different amounts of particles.

5.2.2 Varying Lattice Resolution

In the case of increasing the lattice resolution, Figure 5.3 shows that the FPS decreases at a polynomial rate, which is expected since the resolution is increased in all three dimensions, yielding a cubic increase in lattice points. The time spent in the LBM kernel increases much faster than all the other program parts, which remain relatively constant, with the exception of the rendering time that interestingly triples between $N = 120$ and $N = 160$. The percentage of time spent in CUDA increases rapidly, as expected. Regarding real-time performance, at 10,000 particles, a lattice resolution up to around just under 160 should maintain over 30 FPS.

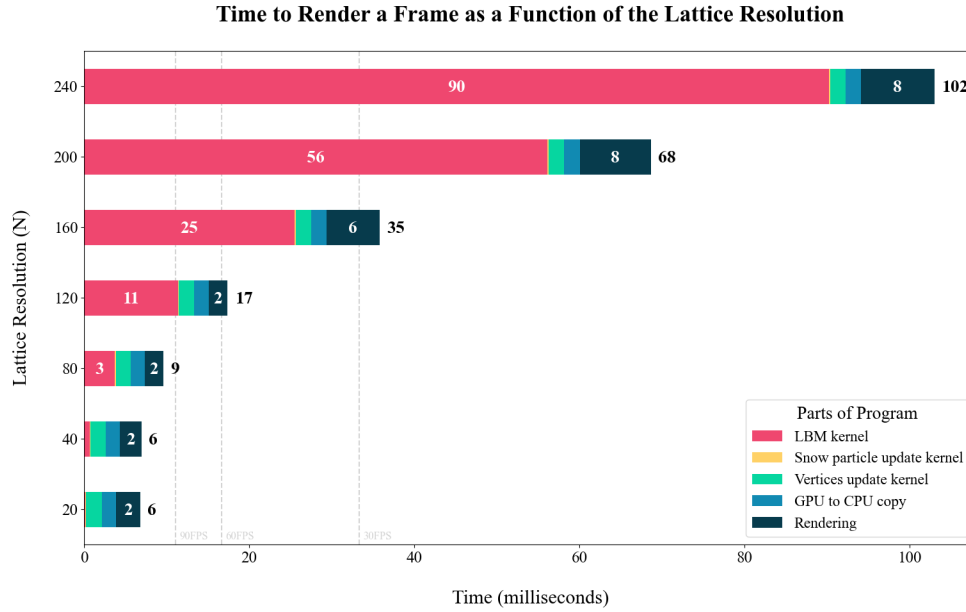


Figure 5.3: A stacked bar graph which presents the time it takes to render a frame, in milliseconds, for different lattice resolutions. The total time is further split into separate time segments for various parts of the program.

As lattice resolution increases, the percentage of CPU memory use remains constant, which is expected since lattice resolution does not affect any data stored on the CPU. In contrast, the GPU percentage of memory use increases linearly. Similarly to the results for varying the number of particles, both GPU and CPU memory use percentages still remain relatively low.

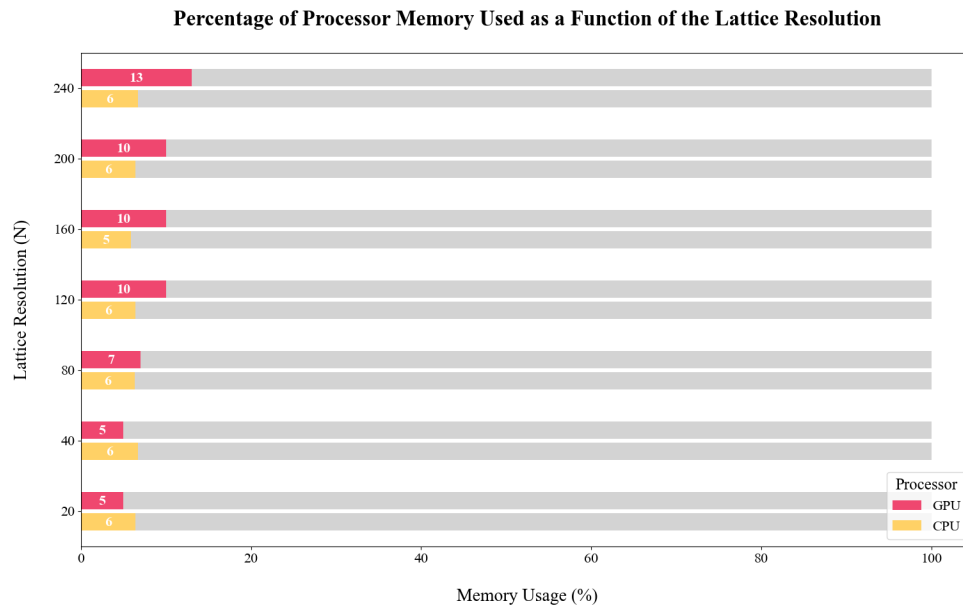


Figure 5.4: A stacked double bar graph which presents the percentage of GPU and CPU memory used for different lattice resolutions.

Chapter 6

Future Work

While this research did bridge an important gap in the area of falling snow simulation, there is still much to be done beyond what this thesis achieved. This chapter describes several general goals for future work and suggests ways to achieve them.

6.1 Towards Further Physical Accuracy

A fundamental notion behind the work this research seeks to accomplish is that improved physical accuracy naturally leads to more accurate visualization, so incorporating as much physical accuracy in the model is important for this project. Moreover, a highly accurate model would have increased value in its application domains, such as in scientific modelling. Areas for improving this aspect are discussed in the following sections. In addition, a general goal is to thoroughly validate the simulation.

6.1.1 More Sophisticated Interactions

In this thesis, **particle-particle interactions** are completely ignored, and **particle-fluid interactions** are relatively simple. Future work should incorporate a more sophisticated model — such as one based on the DEM, which has also been employed in the area of snow visualization — for simulating these interactions [1]. In fact, there is much existing work [13, 14, 11, 12] in the area of integrating **particle transport** with the LBM that could be applied to the specific use case of snow.

Another direction for this research is to model **material exchange** between snow particles — modelled as clumps of snowflakes — upon collision. When snow clumps collide, there are naturally very small-scale interactions between the individual colliding snowflakes making up the snow clumps. Modelling these interactions would increase the model’s physical accuracy, furthering this work’s goals.

6.1.2 An Improved Wind Field

Section 3.2 outlined a subgrid model, the Smagorinsky turbulence model, which would enable wind to be modelled accurately, yet this was not implemented. Immediate future work should integrate this component into the simulation.

Moreover, the wind field generated in this work is relatively simple; it could most certainly be improved. Wang et al. [4] describe various techniques they use to make the wind field more dynamic, as it would be in the physical world; they add a random component to the global wind velocity, change the particle density along different directions at lattice points, and introduce an **acceleration constant** [4].

There are also many constraints on the model for simplicity, such as ensuring the extent and lattice resolution in each dimension are equal. Additionally, the model does not adhere to its governing formulas due to instability, as previously described in Section 4.1.2. Making the wind field more configurable and having parameters actually influence the model would be an important — and high-priority — modification.

6.1.3 More Accurate Particle Updating

As Section 3.5 details, when applying the wind field to the snow particles, inertia is ignored, and calculating the position offset of snow particles is simplified. Both of these should be remedied in future work; for example, a **backwards-integration method** could be used. Indeed, Moeslund et al. [6] do this by using a second-order Runge-Kutta integration. Furthermore, the gravity constant should only account for gravity, and any other forces that are merged with it in this simulation — including drag, buoyancy, and lift [6] — should be considered separately for more accuracy.

6.2 Towards Further High Fidelity Visualization

This thesis largely focused on the physical accuracy of the model and the acceleration of the code on the GPU so there was no consideration of effective visualization beyond the modified Moeslund et al. [6] algorithm outlined in Section 2.2. Improving the physical accuracy naturally improves fidelity, but additional rendering techniques should also be employed. The shader should most definitely be improved beyond the basic Phong model, and the lighting would benefit from, for example, **ray-tracing**²⁶ or **intra-particle lighting**.

Another direction to explore is to compare Moeslund et al.’s [6] snow generation model to Wong and Fu’s, which models snow as a series of concentric, semitransparent, textured shells created by sampling randomly grown ice crystals [52]. Its use of ice crystal growth simulation for generating snow clumps is more physically realistic than Moeslund et al.’s [6] use of triangles, and implementing this model in a way that still maintains the physical properties of the ice crystals in the simulation could be promising.

6.3 Optimization

As with much code, there is always room for improvement regarding optimization. Tran et al. [43] suggest additional optimization strategies — improving **cache**²⁷ **locality**²⁸ and thread parallelism by applying **tiling optimization**²⁹ and reducing **register**³⁰ use per thread, and removing **branch divergence**³¹ — alongside those described by Delbosc et al. [42] for implementing the LBM on the GPU, and including these is a natural next step. The performance of the simulation updating can also be improved by directly integrating OpenGL with CUDA to speed up functionality such as data transfer by keeping all data on the GPU [90].

²⁶A technique for modelling light by tracing light rays [85].

²⁷Memory that stores data so that future requests for it are faster [86].

²⁸The tendency to access the same memory repetitively or to access memory such that the retrieved data elements are near each other [87].

²⁹Ordering memory accesses such that the number of cache hits — meaning the data is found and does not need to be retrieved — is maximized [88].

³⁰Memory which stores information used by a single thread [89].

³¹When flow control instructions on the GPU cause threads running in parallel to diverge [43].

Alongside these strategies, other general practices for GPU programming could also be implemented, such as **loop unrolling** and minimizing branch divergence.

Furthermore, the snow generation in this work runs entirely on the CPU and results in a longer startup time as the number of particles increases. Since the algorithm used is extremely parallelizable, leveraging the GPU to implement this process would improve the startup time drastically.

6.4 Additional Experimentation

The conducted experiments yielded promising results, but also raised questions and revealed areas for improvement. More thorough experimentation should be conducted, and identified faults should be remedied.

After a high fidelity visualization exists, a **qualitative study** should be conducted to determine its actual effectiveness by evaluating its **perceived realness**. Particularly, how realistic the simulation needs to be for human perception and the minimal required resolution should be investigated.

6.5 Testing Suite

At this point, there is no comprehensive testing. A testing suite should be implemented for regression testing and validation.

Chapter 7

Conclusions

Altogether, this research is a small step towards a massively parallel implementation of falling snow which is physically accurate while still visually impressive. By implementing the wind field using the GPU-accelerated LBM and integrating Moeslund et al.'s [6] realistic snow generation model, a gap in the field has been loosely filled. The addition of this research to the existing work on falling snow is also significant as the LBM is an improvement over the commonly used Navier-Stokes equations, as well as Wang et al.'s [4] D3Q15 LBM implementation. Furthermore, its parallelization and resulting real-time capabilities allow it to be used for many more applications, such as game development, AR, and VR, making it more advantageous compared to prior falling snow simulations. What this thesis outlines is also very transferable to simulating other fluids and modelling particle transport in a fluid. With an abundance of directions for future work, this research provides many opportunities to be applied in titillating domains.

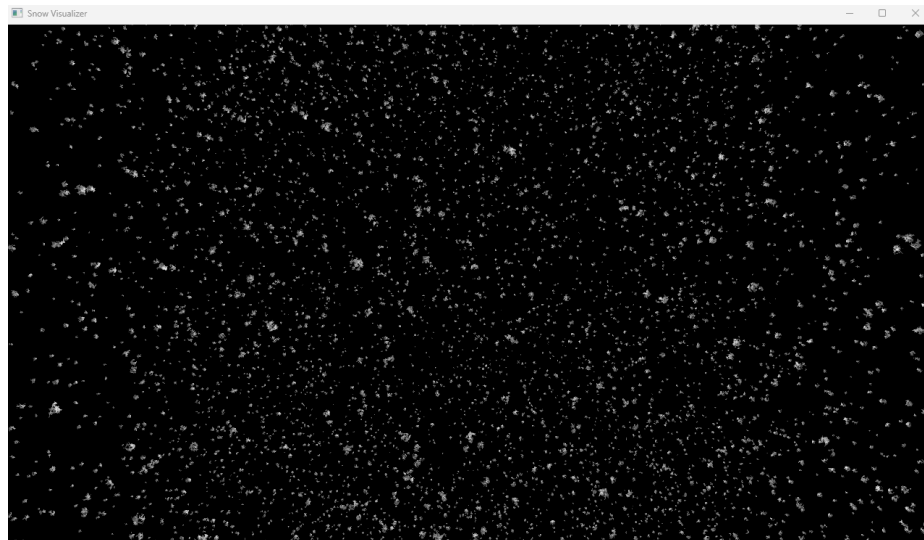


Figure 7.1: A screenshot of the simulation window.

Bibliography

- [1] P. Goswami. Snow and ice animation methods in computer graphics. *Computer Graphics Forum*, 43(2), 2024.
- [2] Wikipedia. OpenGL, 2026. Available at <https://en.wikipedia.org/wiki/OpenGL> (accessed July 15, 2026).
- [3] Wikipedia. CUDA, 2026. Available at <https://en.wikipedia.org/wiki/CUDA> (accessed July 15, 2026).
- [4] C. Wang, Z. Wang, T. Xia, and Q. Peng. Real-time snowing simulation. *The Visual Computer: International Journal of Computer Graphics*, 22(5):315–323, 2006.
- [5] K. Muraoka and N. Chiba. Visual simulation of snowfall, snow cover and snowmelt. In *Seventh International Conference on Parallel and Distributed Systems: Workshops*, pages 187–194. IEEE, 2000.
- [6] T. B. Moeslund, C. B. Madsen, M. Aagaard, and D. Lerche. Modeling falling and accumulating snow. In *Second International Conference on Vision, Video and Graphics, VVG 2005*, pages 61–68. European Association for Computer Graphics, 2005.
- [7] Wikipedia. Vortex, 2026. Available at <https://en.wikipedia.org/wiki/Vortex> (accessed July 15, 2026).
- [8] T.-G. Cowling S. Chapman. *The Mathematical Theory on Non-Uniform Gases*. Cambridge University Press, Cambridge, UK, 1970.
- [9] M. F. Barad, J. G. Kocheemoolayil, and C. C. Kiris. Lattice Boltzmann and Navier-Stokes cartesian CFD approaches for airframe noise predictions. In *23rd AIAA Computational Fluid Dynamics Conference 2017*, page 2276–2291. American Institute for Aeronautics and Astronautics, 2017.
- [10] X. Wei, Y. Zhao, Z. Fan, W. Li, S. Yoakum-Stover, and A. Kaufman. Blowing in the wind. In *Proceedings of the 2003 ACM SIGGRAPH/Eurographics symposium on Computer animation*, page 75–85. Eurographics Association, 2003.
- [11] F. Bazdidi-Tehrani and M. S. Sargazizadeh. Large eddy simulation of flow field and particle dispersion in a ventilated model room using a parallel lattice Boltzmann method. *Aerosol Science and Technology*, 57(6):577–595, 2023.

- [12] M. S. Sargazizadeh and F. Bazdidi-Tehrani. Flow field and particle dispersion in a naturally ventilated model building: Application of large eddy simulation based on parallel lattice Boltzmann method. *Sustainable Cities and Society*, 106:105413, 2024.
- [13] Y. Feng, K. Han, and D. R. J. Owen. Combined three-dimensional lattice Boltzmann method and discrete element method for modeling fluid-particle interactions with experimental assessment. *International Journal for Numerical Methods in Engineering*, 81(2):229–245, 2009.
- [14] L. Ding and A. C. K. Lai. An efficient lattice Boltzmann model for indoor airflow and particle transport. *Journal of Aerosol Science*, 63:10–24, 2013.
- [15] J. Herrera, T. Hädrich, W. Pałubicki, D. T. Banuti, S. Pirk, and D. L. Michels. Weatherscapes: nowcasting heat transfer and water continuity. *ACM Transactions on Graphics*, 40(6), 2021.
- [16] R. Bridson. *Fluid Simulation for Computer Graphics*. CRC Press, Boca Raton, FL, USA, 2015.
- [17] T. Hädrich, M. Makowski, W. Pałubicki, D. T. Banuti, S. Pirk, and D. L. Michels. Stormscapes: Simulating cloud dynamics in the now. *ACM Transactions on Graphics*, 39(6), 2020.
- [18] R. Eidissen. Utilizing GPUs for real-time visualization of snow. Master’s thesis, Norwegian University of Science and Technology, Trondheim Ålesund Gjøvik, Norway, 2009.
- [19] I. Saltvik. Parallel methods for real-time visualization of snow. Master’s thesis, Norwegian University of Science and Technology, Trondheim Ålesund Gjøvik, Norway, 2006.
- [20] M. Griebel, T. Dornseifer, and T. Neunhoeffler. *Numerical simulation in fluid dynamics: a practical introduction*. Society for Industrial and Applied Mathematics, Philadelphia, PA, USA, 1998.
- [21] J. Stam. Stable fluids. In *Proceedings of the 26th Annual Conference on Computer Graphics and Interactive Techniques*, page 121–128. ACM Press/Addison-Wesley Publishing Co., 1999.
- [22] R. Fedkiw, J. Stam, and H. W. Jensen. Visual simulation of smoke. In *Proceedings of the 28th Annual Conference on Computer Graphics and Interactive Techniques*, page 15–22. Association for Computing Machinery, 2001.
- [23] J. Stam. Real-time fluid dynamics for games. 2003.

- [24] Y. Liu, X. Liu, and E. Wu. Real-time 3d fluid simulation on GPU with complex obstacles. In *12th Pacific Conference on Computer Graphics and Applications, 2004. PG 2004. Proceedings*, pages 247–256. IEEE, 2004.
- [25] R. Barrett, M. Berry, T. F. Chan, J. Demmel, J. M. Donato, J. Dongarra, V. Eijkhout, R. Pozo, C. Romine, and H. Van der Vorst. *Templates for the Solution of Linear Systems: Building Blocks for Iterative Methods*. Society for Industrial and Applied Mathematics, Philadelphia, PA, USA, 1994.
- [26] T. Vik, A. C. Elster, and T. Hallgren. Real-time visualization of smoke through parallelizations. In *Parallel Computing*, volume 13 of *Advances in Parallel Computing*, pages 371–378. North-Holland, 2004.
- [27] I. Saltvik, A. Elster, and H. Nagel. Parallel methods for real-time visualization of snow. In *Proceedings of the 8th International Conference on Applied Parallel Computing: State of the Art in Scientific Computing*, pages 218–227. Springer-Verlag, 2006.
- [28] K. Muraoka, N. Chiba, T. Takahashi, and N. Saito. Image generation method for seasonal mountain scener. *Image Generation Method for Seasonal Mountain Scener*, 22(5):466–474, 1993. In Japanese.
- [29] J. Takahashi, H. Takahashi, and N. Chiba. Image synthesis of flickering scenes including simulated flames. *IEICE Transactions on Information and Systems*, E80(11):1102–1108, 1997.
- [30] NeHe Productions. Billboarding how to, 2014. Available at https://nehe.gamedev.net/article/billboarding_how_to/18011/ (accessed July 15, 2026).
- [31] Wikipedia. Hyper-threading, 2026. Available at <https://en.wikipedia.org/wiki/Hyper-threading> (accessed July 15, 2026).
- [32] S. Andreasson, L. Östergaard, and P. Goswami. Parallel spatiotemporally adaptive DEM-based snow simulation. *Proceedings of the ACM on Computer Graphics and Interactive Techniques*, 7(3), 2024.
- [33] P. Goswami, A. Nordin, and S. Nylén. Iterative discrete element solver for efficient snow simulation. In *22nd Eurographics Symposium on Parallel Graphics and Visualization*, pages 19–30. Eurographics Association, 2022.
- [34] P. Goswami, C. Markowicz, and A. Hassan. Real-time particle-based snow simulation on the GPU. In *19th Eurographics Symposium on Parallel Graphics and Visualization*, pages 49–58. Eurographics Association, 2019.
- [35] N. Mukai, Y. Eto, and Y. Chang. Representation method of snow splitting and sliding on a roof. In *5th International Conference on “Advances in Engineering and Technology”*, pages 100–103, 2017.

- [36] C. Jiang, C. Schroeder, J. Teran, A. Stomakhin, and A. Selle. The material point method for simulating continuum materials. In *ACM SIGGRAPH 2016 Courses*. Association for Computing Machinery, 2016.
- [37] A. Stomakhin, C. Schroeder, L. Chai, J. Teran, and A. Selle. A material point method for snow simulation. *ACM Transactions on Graphics*, 32(4), 2013.
- [38] T. Takahashi, I. Fujishiro, and T. Nishita. Visual simulation of compressible snow with friction and cohesion. In *Proceedings of NICOGRAPH International*, pages 35–42, 2014.
- [39] C. Gissler, A. Henne, S. Band, A. Peer, and M. Teschner. An implicit compressible SPH solver for snow simulation. *ACM Transactions on Graphics*, 39(4), 2020.
- [40] C. Körner, T. Pohl, U. Rüde, N. Thürey, and T. Zeiser. *Parallel Lattice Boltzmann Methods for CFD Applications*, volume 51, pages 439–466. Springer-Verlag, Germany, 2006.
- [41] C. Schepke, N. Maillard, and P. Navaux. Parallel lattice Boltzmann method with blocked partitioning. *International Journal of Parallel Programming*, 37(6):593–611, 2009.
- [42] N. Delbosc, J. L. Summers, A. I. Khan, N. Kapur, and C. J. Noakes. Optimized implementation of the lattice Boltzmann method on a graphics processing unit towards real-time fluid simulation. *Computers & Mathematics with Applications*, 67(2):462–475, 2014.
- [43] N.-P. Tran, M. Lee, and S. Hong. Performance optimization of 3D lattice Boltzmann flow solver on a GPU. *Scientific Programming*, 2017.
- [44] C. Obrecht F. Kuznik, B. Tourancheau, and J.-J. Roux. A new approach to the lattice Boltzmann method for graphics processing units. *Computers & Mathematics with Applications*, 61(12):3628–3638, 2011.
- [45] P. R. Rinaldi, E. A. Dari, M. J. Vénere, and A. Clausse. A lattice-Boltzmann solver for 3D fluid simulation on GPU. *Simulation Modelling Practice and Theory*, 25:163–171, 2012.
- [46] A. von Bernuth, G. Volk, and O. Bringmann. Simulating photo-realistic snow and fog on existing images for enhanced CNN training and evaluation. In *2019 IEEE Intelligent Transportation Systems Conference (ITSC)*, pages 41–46. IEEE, 2019.
- [47] M. S. Langer, L. Zhang, A. W. Klein, A. Bhatia, J. Pereira, and D. Rekhi. A spectral-particle hybrid method for rendering falling snow. In *Proceedings of the Fifteenth Eurographics Conference on Rendering Techniques*, page 217–226, Goslar, Germany, 2004. Eurographics Association.

- [48] T. Hinks. Animating wind-driven snow buildup using an implicit approach. Master's thesis, Linköping University, Norrköping, Sweden, 2006.
- [49] C. Zou, X. Xie, and G. Zhao. Algorithm for generating snow based on GPU. In *Proceedings of the Second International Conference on Internet Multimedia Computing and Service*, page 199–202. Association for Computing Machinery, 2010.
- [50] Wikipedia. Bézier curve, 2026. Available at https://en.wikipedia.org/wiki/B%C3%A9zier_curve (accessed July 15, 2026).
- [51] T. Kim and M. C. Lin. Visual simulation of ice crystal growth. In *Proceedings of the 2003 ACM SIGGRAPH/Eurographics symposium on Computer animation*, page 86–97. Eurographics Association, 2003.
- [52] S.-K. Wong and I.-T. Fu. Hybrid-based snow simulation and snow rendering with shell textures. *Computer Animation and Virtual Worlds*, 26(3-4):413–421, 2015.
- [53] D. Micciancio. 1: Introduction to lattices, 2010. Available at <https://cseweb.ucsd.edu/classes/wi10/cse206a/lec1.pdf> (accessed July 15, 2026).
- [54] G. R. McNamara and G. Zanetti. Use of the Boltzmann equation to simulate lattice-gas automata. *Physical Review Letters*, 61:2332–2335, 1988.
- [55] Wikipedia. Lattice gas automaton, 2025. Available at https://en.wikipedia.org/wiki/Lattice_gas_automaton (accessed July 15, 2026).
- [56] D. Wolf-Gladrow. *Lattice-Gas Cellular Automata and Lattice Boltzmann Models - An Introduction*. Springer-Verlag, Germany, 2000.
- [57] P. L. Bhatnagar, E. P. Gross, and M. Krook. A model for collision processes in gases. I. small amplitude processes in charged and neutral one-component systems. *Physical Review*, 94(3):511–525, 1954.
- [58] Wikipedia. Thermodynamic equilibrium, 2026. Available at https://en.wikipedia.org/wiki/Thermodynamic_equilibrium (accessed July 15, 2026).
- [59] Wikipedia. Reynolds number, 2026. Available at https://en.wikipedia.org/wiki/Reynolds_number (accessed July 15, 2026).
- [60] D. Troyer. Kinematic viscosity explained, 2025. Available at <https://www.machinerylubrication.com/Read/294/absolute-kinematic-viscosity> (accessed July 15, 2026).

- [61] S. Hou, J. D. Sterling, S. Chen, and G. D. Doolen. A lattice Boltzmann subgrid model for high Reynolds number flows. *Fields Institute Communications*, 6, 1996.
- [62] J. Smagorinsky. General circulation experiments with the primitive equations: I. the basic experiment. *Monthly Weather Review*, 91(3):99–164, 1963.
- [63] ScienceDirect. Stress tensor, 2026. Available at <https://www.sciencedirect.com/topics/physics-and-astronomy/stress-tensor> (accessed July 15, 2026).
- [64] ScienceDirect. Inlet flow, 2026. Available at <https://www.sciencedirect.com/topics/earth-and-planetary-sciences/inlet-flow> (accessed July 15, 2026).
- [65] S. C. Wetstein. Implementing the lattice-Boltzmann method: A research on boundary condition techniques, 2014. Available at https://filelist.tudelft.nl/TNW/Afdelingen/Radiation%20Science%20and%20Technology/Research%20Groups/RPNM/Publications/BSc_Suzanne_Wetstein.pdf (accessed July 15, 2026).
- [66] Wikipedia. No-slip condition, 2026. Available at https://en.wikipedia.org/wiki/No-slip_condition (accessed July 15, 2026).
- [67] U. Alim, A. Entezari, and T. Möller. The lattice-Boltzmann method on optimal sampling lattices. *IEEE Transactions on Visualization and Computer Graphics*, 15(4):630–641, 2009.
- [68] D. Barker. Lattice Boltzmann parameterization, 2025. Available at <https://dbarker.uk/posts/lbm-parameterization/> (accessed July 15, 2026).
- [69] Wikipedia. Mach number, 2026. Available at https://en.wikipedia.org/wiki/Mach_number (accessed July 15, 2026).
- [70] Wikipedia. Drag (physics), 2026. Available at [https://en.wikipedia.org/wiki/Drag_\(physics\)](https://en.wikipedia.org/wiki/Drag_(physics)) (accessed August 26, 2026).
- [71] Wikipedia. Buoyancy, 2026. Available at <https://en.wikipedia.org/wiki/Buoyancy> (accessed August 26, 2026).
- [72] Wikipedia. Lift (force), 2026. Available at [https://en.wikipedia.org/wiki/Lift_\(force\)](https://en.wikipedia.org/wiki/Lift_(force)) (accessed August 26, 2026).
- [73] Wikipedia. Inertia, 2026. Available at <https://en.wikipedia.org/wiki/Inertia> (accessed July 15, 2026).
- [74] Wikipedia. Speed of sound, 2026. Available at https://en.wikipedia.org/wiki/Speed_of_sound (accessed July 15, 2026).

- [75] P. Evans. Properties of air at atmospheric pressure, 2015. Available at <https://theengineeringmindset.com/properties-of-air-at-atmospheric-pressure/> (accessed July 15, 2026).
- [76] Wikipedia. Memory bandwidth, 2026. Available at https://en.wikipedia.org/wiki/Memory_bandwidth (accessed July 15, 2026).
- [77] N. Stawinoga and T. Field. Predictable thread coarsening. *ACM Transactions on Architecture and Code Optimization*, 15(2):1–26, 2018.
- [78] D. Tichonov. CUDA series: Memory and allocation, 2024. Available at <https://medium.com/@dmitrijtichonov/cuda-series-memory-and-allocation-fce29c965d37> (accessed July 15, 2026).
- [79] NVIDIA. 1.2. programming model, 2026. Available at <https://docs.nvidia.com/cuda/cuda-programming-guide/01-introduction/programming-model.html> (accessed July 15, 2026).
- [80] Wikipedia. Thread block (CUDA programming), 2026. Available at [https://en.wikipedia.org/wiki/Thread_block_\(CUDA_programming\)](https://en.wikipedia.org/wiki/Thread_block_(CUDA_programming)) (accessed July 15, 2026).
- [81] OpenGL Wiki. Vertex specification, 2024. Available at https://wikis.khronos.org/opengl/Vertex_Specification#Vertex_Array_Object (accessed August 26, 2026).
- [82] Wikipedia. Phong shading, 2026. Available at https://en.wikipedia.org/wiki/Phong_shading (accessed July 15, 2026).
- [83] Wikipedia. Frame rate, 2026. Available at https://en.wikipedia.org/wiki/Frame_rate (accessed July 15, 2026).
- [84] IrisVR. The importance of frame rates, 2024. Available at <https://help.irisvr.com/hc/en-us/articles/215884547-The-Importance-of-Frame-Rates> (accessed July 15, 2026).
- [85] Wikipedia. Ray tracing (graphics), 2026. Available at [https://en.wikipedia.org/wiki/Ray_tracing_\(graphics\)](https://en.wikipedia.org/wiki/Ray_tracing_(graphics)) (accessed July 15, 2026).
- [86] Wikipedia. Cache (computing), 2026. Available at [https://en.wikipedia.org/wiki/Cache_\(computing\)](https://en.wikipedia.org/wiki/Cache_(computing)) (accessed August 26, 2026).
- [87] Wikipedia. Locality of reference, 2026. Available at https://en.wikipedia.org/wiki/Locality_of_reference (accessed August 26, 2026).
- [88] B. Brett. Efficient use of tiling, 2016. Available at <https://www.intel.com/content/www/us/en/developer/articles/technical/efficient-use-of-tiling.html> (accessed August 26, 2026).

- [89] Modal. What are registers?, 2026. Available at <https://modal.com/gpu-glossary/device-software/registers> (accessed August 26, 2026).
- [90] NVIDIA. 4.19. CUDA interoperability with APIs, 2026. Available at <https://docs.nvidia.com/cuda/cuda-programming-guide/04-special-topics/graphics-interop.html> (accessed July 15, 2026).